

گزارش یادگیری هفته پنجم: مفاهیم ابری، استقرار و تحویل نرم افزار

مقدمه

پس از چهار هفته پربار که در آن‌ها از مبانی پایتون و FastAPI تا پایگاه‌های داده پیشرفته، جستجوی هوشمند و امنیت وب را فرا گرفتیم، در هفته پنجم و پایانی این دوره، تمرکز خود را بر روی آخرین و حیاتی‌ترین مرحله از چرخه توسعه نرم‌افزار می‌گذاریم: استقرار (Deployment) و تحویل (Delivery). هدف این هفته آن است که بیاموزیم چگونه نرم‌افزاری که با دقت ساخته و تست کرده‌ایم را به دست کاربران واقعی برسانیم، آن را در محیطی پایدار و مقیاس‌پذیر اجرا کنیم و با مفاهیم مدرن زیرساخت ابری آشنا شویم.

۱. مدل‌های سرویس رایانش ابری

(Cloud Computing Service Models)

برای درک چگونگی میزبانی نرم‌افزارهای امروزی، ابتدا باید با سطوح مختلف انتزاع در سرویس‌های ابری آشنا شویم. این مدل‌ها مشخص می‌کنند که چه میزان از مسئولیت مدیریت زیرساخت بر عهده ما (توسعه‌دهنده) و چه میزان بر عهده ارائه‌دهنده سرویس ابری است. برای درک بهتر، از تشبیه «پیتزا» استفاده می‌کنیم:

- مدل سنتی (OnPremise): در این مدل، همه چیز بر عهده خودتان است. شما باید زمین را بخرید، گندم بکارید، آسیاب کنید، خمیر درست کنید، پنیر بسازید و در نهایت پیتزا را بپزید. به بیان فنی، شما مسئول خرید سرور، نصب سیستم‌عامل، شبکه‌بندی، نصب پایگاه داده، پیکربندی محیط اجرا و در نهایت اجرای نرم‌افزار خود هستید. این مدل بیشترین کنترل و بیشترین مسئولیت را به شما می‌دهد.

- زیرساخت به عنوان سرویس (IaaS Infrastructure as a Service): در این مدل، ارائه‌دهنده ابری، مواد اولیه آماده (خمیر و پنیر) را در اختیار شما می‌گذارد و شما خودتان

پیتزا را درست می‌کنید. به بیان فنی، شما یک ماشین مجازی با سیستم‌عامل نصب‌شده تحویل می‌گیرید و دیگر نگران سخت‌افزار، برق و خنک‌سازی نیستید، اما همچنان مسئول نصب و پیکربندی هر چیزی که روی آن اجرا می‌شود (از جمله زمان اجرای زبان برنامه‌نویسی و پایگاه داده) هستید. آمازون EC2 و دیجیتال اوشن نمونه‌های معروف IaaS هستند.

- پلتفرم به عنوان سرویس (PaaS Platform as a Service): این مدل یک آشپزخانه کامل و آماده در اختیار شما می‌گذارد. شما فقط مواد اولیه (کد) خود را می‌آورید و پیتزا را می‌پزید. به بیان فنی، شما فقط کد برنامه خود را ارائه می‌دهید و پلتفرم ابری به طور خودکار آن را می‌سازد، پیکربندی می‌کند و اجرا می‌نماید. نگرانی‌هایی مانند مدیریت سیستم‌عامل، وصله‌های امنیتی و مقیاس‌پذیری توسط پلتفرم مدیریت می‌شود. Heroku و سرویس ابری ایرانی Liara مثال‌هایی از این مدل هستند.

- نرم‌افزار به عنوان سرویس (SaaS Software as a Service): در این سطح، شما حتی زحمت پختن پیتزا را هم نمی‌کشید؛ آن را به صورت آماده سفارش می‌دهید و نوش جان می‌کنید. به بیان فنی، شما از یک نرم‌افزار کامل و آماده استفاده می‌کنید که توسط ارائه‌دهنده در ابر مدیریت می‌شود و هیچ مسئولیتی در قبال زیرساخت یا کد آن ندارید. جیمیل، گوگل داکس و ترلو نمونه‌های آشنای SaaS هستند.

یک مدل تخصصی دیگر که در این اکوسیستم بسیار مهم است، پایگاه داده به عنوان سرویس (DBaaS) نام دارد. در این مدل، شما یک نمونه پایگاه داده (مثلاً PostgreSQL) را تحویل می‌گیرید که تمام امور نگهداری، پشتیبان‌گیری، به‌روزرسانی و مقیاس‌پذیری آن توسط ارائه‌دهنده مدیریت می‌شود و شما تنها از آن استفاده می‌کنید. سرویس‌هایی مانند Amazon RDS یا MongoDB Atlas از این دسته هستند.

۲. آشنایی با ارائه‌دهندگان ابری ایرانی

- با توجه به نیاز به سرعت و پینگ پایین برای کاربران داخل ایران و همچنین تسهیل فرآیندهای پرداخت و پشتیبانی، سرویس‌های ابری ایرانی گزینه‌های بسیار مناسبی برای استقرار پروژه‌ها هستند. در این میان با سه نام مهم آشنا شدیم:
- Liara (لیارا): یک پلتفرم ابری مدرن و قدرتمند ایرانی از نوع PaaS است. لیارا فرآیند دیپلوی را بسیار ساده کرده است؛ شما کد خود را روی گیت‌هاب پوش می‌کنید و لیارا به صورت خودکار آن را می‌سازد و اجرا می‌کند. پشتیبانی عالی از Docker، اعطای دامنه رایگان، فعال‌سازی خودکار HTTPS و امکان نصب سرویس‌های جانبی مانند PostgreSQL و Redis از ویژگی‌های برجسته آن است.
- Hamravesht (همروش): یک پلتفرم ابری دیگر است که رویکردی مشابه دارد و با پشتیبانی از Docker، مانیتورینگ و مقیاس‌پذیری خودکار، گزینه مناسبی برای پروژه‌های حرفه‌ای محسوب می‌شود.
- Abr Arvan (ابر آروان): این شرکت عمدتاً به عنوان ارائه‌دهنده IaaS و CDN شناخته می‌شود. سرویس ابرک آن، سرورهای ابری با دسترسی کامل (Root Access) ارائه می‌دهد و شبکه توزیع محتوا (CDN) آن برای افزایش سرعت بارگذاری محتوای استاتیک در سراسر ایران بسیار محبوب است.

۳. مفاهیم مدرن استقرار نرم افزار

فراتر از انتخاب یک پلتفرم ابری، آشنایی با مفاهیم و ابزارهای مدرن استقرار برای یک توسعه دهنده بک اند ضروری است:

۳.۱ کانتینر سازی با Docker

داکر یک پلتفرم برای بسته بندی، توزیع و اجرای نرم افزار در واحدهایی استاندارد به نام کانتینر است. یک کانتینر شامل کد، کتابخانه ها و تمام وابستگی های مورد نیاز برای اجرای نرم افزار است. مهم ترین مزیت داکر حل مشکل معروف "روی سیستم من که کار می کرد!" است؛ یک کانتینر داکر روی هر سیستم عاملی که داکر نصب باشد، دقیقاً به یک شکل اجرا می شود. فایل `Dockerfile` مانند یک دستورالعمل آشپزی، نحوه ساخت یک Image (تصویر) را مشخص می کند و `Docker Compose` امکان اجرای چند کانتینر در کنار هم (مانند برنامه، پایگاه داده و Redis) را فراهم می سازد.

۳.۲ ادغام و استقرار مداوم (CI/CD)

CI/CD مخفف Continuous Integration و Continuous Delivery/Deployment است. این یک روش شناسی برای خودکار سازی فرآیند تست و انتشار نرم افزار است. به این صورت که: ادغام مداوم (CI): هر بار که توسعه دهنده کدی را در مخزن گیت هاب ادغام می کند، یک سرور خودکار (مانند GitHub Actions) کد را گرفته، آن را می سازد و تمام تست ها را روی آن اجرا می کند. این کار باعث می شود خطاها سریع تر کشف شوند. استقرار مداوم (CD): اگر مرحله ادغام با موفقیت طی شود، همان سرور خودکار، کد تست شده را مستقیماً روی سرور اصلی (Production) مستقر می کند. این فرآیند سرعت و اطمینان انتشار نرم افزار را به شدت افزایش می دهد.

۳.۳ اصول ۱۲ عاملی (The TwelveFactor App)

این یک متدولوژی برای ساخت برنامه های ابری مدرن (Software as a Service) است که توسط مهندسان Heroku تدوین شده. برخی از مهم ترین اصول آن عبارتند از:

- کدبیس (Codebase): یک کدبیس واحد در سیستم کنترل نسخه، که می‌تواند چندین بار مستقر شود.
- وابستگی‌ها (Dependencies): تمام وابستگی‌ها باید به طور صریح و ایزوله (مثلاً در فایل `requirements.txt`) اعلام شوند.
- پیکربندی (Config): هر آنچه بین محیط‌های مختلف (توسعه، تست، تولید) تغییر می‌کند (مانند رمزهای عبور و آدرس‌های اتصال) باید در متغیرهای محیطی (Environment Variables) ذخیره شود و هرگز در کد هاردکد نشود.
- فرآیندها (Processes): برنامه باید به صورت فرآیندهای بی‌حالت (Stateless) اجرا شود و هیچ داده‌ای را در حافظه محلی یا دیسک ذخیره نکند. این اصل با معماری‌هایی مانند JWT (که سرور حالت نشست را ذخیره نمی‌کند) همخوانی کامل دارد.
- لاگ‌ها (Logs): برنامه باید لاگ‌ها را به عنوان یک جریان رویداد (Event Stream) در خروجی استاندارد چاپ کند و وظیفه مدیریت و ذخیره‌سازی لاگ‌ها را به سرویس‌های خارجی بسپارد.

۳.۴ متغیرهای محیطی (Environment Variables)

همانطور که در اصل سوم اپلیکیشن ۱۲عاملی ذکر شد، جداسازی پیکربندی از کد یک اصل حیاتی است. در محیط Production، ما هرگز اطلاعات حساس مانند کلیدهای مخفی، رشته اتصال پایگاه داده و توکن‌های API را در کد قرار نمی‌دهیم. در عوض، این مقادیر را به عنوان متغیرهای محیطی در سیستم میزبان (مثلاً در پنل لیا یا داشبورد سرور ابری) تعریف می‌کنیم و برنامه آن‌ها را از محیط می‌خواند. این کار امنیت را بالا می‌برد و انتقال بین محیط‌های مختلف را ساده می‌کند.

۳.۵ ثبت رویدادها و پایش (Logging & Monitoring)

پس از استقرار نرم‌افزار، مشاهده عملکرد آن و دیباگ مشکلات احتمالی حیاتی است.

- Logging (ثبت رویدادها): به معنای ذخیره‌سازی تمام اتفاقاتی است که در برنامه رخ می‌دهد، از پردازش موفق یک درخواست گرفته تا وقوع یک خطای بحرانی. یک سیستم

لاگینگ خوب باید امکان جستجو و فیلتر کردن لاگ‌ها را فراهم کند. ابزار ELK Stack که در هفته‌های گذشته با آن آشنا شدیم، یک راه‌حل کامل برای مدیریت لاگ‌ها است.

- Monitoring (پایش): به معنای نظارت مداوم بر سلامت و عملکرد سیستم است. معیارهایی مانند میزان مصرف CPU، حافظه RAM، تعداد درخواست‌ها در ثانیه (RPS) و نرخ خطاها به صورت زنده رصد می‌شوند. Grafana که پیشتر معرفی شد، یکی از بهترین ابزارها برای ساخت داشبوردهای مانیتورینگ است. پایش صحیح به ما امکان می‌دهد پیش از آنکه کاربران متوجه مشکلی شوند، ما آن را شناسایی و برطرف کنیم.

جمع‌بندی

هفته پنجم، پازل سفر یادگیری ما را تکمیل کرد. ما از دنیای منطق و کد فراتر رفتیم و وارد قلمروی تحویل نرم‌افزار شدیم. آموختیم که یک نرم‌افزار هرچقدر هم که عالی نوشته شده باشد، تا زمانی که به صورت پایدار و امن در دسترس کاربران قرار نگیرد، ارزش خود را به طور کامل نشان نمی‌دهد. با درک مدل‌های سرویس ابری، با ارائه‌دهندگان داخلی برای استقرار پروژه‌هایمان آشنا شدیم، به اهمیت کانتینرسازی با Docker پی بردیم و فلسفه CI/CD و ۱۲فاکتور را به عنوان نقشه راه ساخت نرم‌افزارهای مدرن و مقیاس‌پذیر در آغوش گرفتیم. این پایان دوره نیست، بلکه آغازی است برای ورود حرفه‌ای به دنیای بی‌انتهای مهندسی نرم‌افزار.